MINISTERUL EDUCAȚIEI, CULTURII ȘI CERCETĂRII

AL REPUBLICII MOLDOVA

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică şi Microelectronică

Departamentul Ingineria Software și Automatică

Smolenschi Petru

Raport

pentru lucrarea de laborator Nr. 1 (Varianta 8)

la cursul de "Structuri de Date și Algoritm"

Verificat:

Guțu Maria, *asist. univ.*

Departamentul Ingineria Software și Automatică,

Facultatea FCIM, UTM

Chișinău – 2026

Scopul lucrării:

Programarea algoritmilor de prelucrare a tablourilor aplicând tehnicile și metodele de sortare prin utilizarea funcțiilor, pointerilor, alocării dinamice a memoriei în limbajul C.

Sarcina (conform variantelor):

Elaborarea unui program in limbajul de programare C unde vom crea un meniu recursiv. Acesta trbuie să cuprindă următoarele funcții in C (cu apelare ulterioară ale acestora în funcție *main()*):

1. *Introducerea valorilor tabloului de la tastatură;*
2. *Completarea tabloului cu valori random;*
3. *Afișarea elementelor tabloului la ecran;*
4. *Sortarea elementelor tabloului conform variantelor:*
    1) Bubble Sort;
    2) Selection Sort;
    3) Insertion Sort;
    4) Merge Sort;
    5) Quick Sort;
    6) Shell Sort;
    7) Counting Sort;
    8) Heap Sort;
    9) Radix Sort;
    10) Comb Sort;
5. *Eliberarea memoriei și ieșirea din program.*

P.S. După fiecare manipulare a datelor să fie prevăzută afișarea rezultatelor ca și concluzie. Algoritmul funcțiilor va fi programat, reieșind din conținutul problemei în două versiuni:

A. cu utilizarea metodei de transmitere a parametricelor funcțiilor prin valoare;

B. cu utilizarea metodei de transmitere a parametricelor funcțiilor prin adresă / Pointeri (parametrul formal va fi un pointer către obiectul corespunzător).

Varianta 8:

1. Se dă un array unidimensional cu elemente de tip integer și un număr

n (condiția trebuie să se verifice în program!), valoarea căruia este citită de la tastatură. Să se afișeze array-ul original și array-ul modificat după fiecare manipulare a datelor din array, afișarea rezultatelor ca și concluzie (De ex.: Nu există numere prime; Nu există așa element cu indexul dat și alte date stipulare în condiția problemei) etc.

A. Dacă primul element, număr negativ, întâlnit în array, se află pe poziție pară, atunci să se sorteze elementele array-ului crescător, aplicând tehnica de sortare HeapSort, altfel să se sorteze elementele array-ului descrescător, aplicând tehnica de sortare CountingSort.

B. Dacă suma elementelor pare a array-ului este mai mică decât media aritmetică a elementelor impare din array, atunci să se sorteze elementele array-ului descrescător, aplicând tehnica de sortare RadixSort, altfel să se sorteze elementele array-ului crescător, aplicând tehnica de sortare CombSort.

C. Dacă există cel puțin două elemente numere prime, atunci să se sorteze elementele array-ului crescător, aplicând tehnica de sortare MergeSort, altfel să se sorteze elementele array-ului descrescător, aplicând tehnica de sortare BubbleSort.

2. Se dă un array bidimensional cu elemente de tip *integer* și un număr natural n ($n \geq 0$), valoarea căruia este citită de la tastatură. Să se afișeze array-ul original și array-ul modificat după fiecare manipulare a datelor din array, afișarea rezultatelor ca și concluzie (De ex.: Nu există numere prime; Nu există așa element cu indexul dat și alte date stipulate în condiția problemei) etc.

A. Dacă produsul elementelor de pe prima coloană este mai mare decât valoarea lui k, declarată ca o constantă globală, atunci să se sorteze crescător elementele de pe diagonala principală a array-ului, aplicând tehnica de sortare QuickSort, altfel să se sorteze descrescător elementele de pe diagonala secundară a array-ului, aplicând tehnica de sortare ShellSort.

B. Dacă elementul minim din array se află cel puțin o dată pe linie impară, atunci să se sorteze crescător elementele liniei în care se află elementul minim, aplicând tehnica de sortare SelectionSort, altfel să se sorteze descrescător elementele de pe coloana în care se află elementul minim, aplicând tehnica de sortare InsertionSort.

Cod:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

// Constanta k care a fost cerută de problema 2A
const int k = 69;

// Regiunea cu funcțiile utile ale programului
#pragma region Utils

// Curatarea ecranului / consolei
void clrscr() {
    // !!! NOTA !!!
    // A SE SCHIMBA COMANDA DIN ARGUMENTUL FUNCTIEI 'system();' IN
DEPENDENTA DE SISTEMUL DE OPERARE
    // WINDOWS: 'system("cls");'
    // UNIX/LINUX/MAC: 'system("clear");'
    system("clear");
    //system("cls");
}

// Afisarea array-ului
void print_arr(int *arr, int *clone, int size) {

    printf("\nArray-ul original: ");
    for(int i = 0; i < size; i++) {
        printf("%d ", clone[i]);
    }

    printf("\nElementele array-ului dupa manipulare: ");
    for(int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }

    printf("\n\n");
}

// Afisare matricei
void print_mat(int *mat, int *clone, int size) {
    printf("\nMatricea originala:\n");
    for(int i = 0; i < size; i++) {
        for(int j = 0; j < size; j++) {
            printf("%d\t", clone[i * size + j]);
        }
```

```
        printf("\n");
    }

    printf("\nElementele matricei dupa manipulare: \n");
    for(int i = 0; i < size; i++) {
        for(int j = 0; j < size; j++) {
            printf("%d\t", mat[i * size + j]);
        }
        printf("\n");
    }

    printf("\n");
}

// Swap
void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

// Functie care valideaza daca variabila 'number' este un numar prim
int isPrime(int number) {

    if(number <= 1) return 0;

    for(int i =  2; i * i <= number; i++) {
        if(number % i == 0) return 0;
    }

    return 1;
}

// Functiile de mai jos cloneaza array-ul si matricea. Ele sunt
folosite la afisarea elementelor originale dupa manipularea lor
void *clone_arr(int *arr, int *clone, int size) {
    for(int i = 0; i < size; i++)
        clone[i] = arr[i];
}

void *clone_mat(int *mat, int *clone, int size) {
    for(int i = 0; i < size; i++)
        for(int j = 0; j < size; j++)
            clone[i * size + j] = mat[i * size + j];
}

// Eliberarea memoriei
void free_mem(int *arr, int *mat) {
```

```
    if(arr != NULL)
        free(arr);

    if(mat != NULL)
        free(mat);

    printf("Memoria a fost eliberata cu succes!\n");
}

// Alocarea memoriei pentru array
int *mem_alloc(int size) {
    int *arr = (int*)malloc(size * sizeof(int));
    if(arr == NULL) {
        printf("Nu s-a putut aloca memorie pentru array!.\n EXITING
PROGRAM...");
        exit(-1);
    }

    return arr;
}

// Alocarea memoriei pentru matrice
int *mem_alloc_mat(int size) {
    int *mat = (int*)malloc(size * size * sizeof(int));
    if(mat == NULL) {
        printf("Nu s-a putut aloca memorie pentru matrice!.\n
EXITING PROGRAM...");
        exit(-1);
    }

    return mat;
}

// Functii care introduce valorile de la tastatura in array si
matrice
void input8_1(int *arr, int size) {
    for(int i = 0; i < size; i++) {
        printf("Dati valoare elementului %d: ", i);
        scanf("%d", &arr[i]);
    }
}

void input8_2(int *mat, int size) {
    for(int i = 0; i < size; i++) {
        for(int j = 0; j < size; j++) {
            printf("Dati valoare elementului de pe pozitia [%d, %d]:
", i, j);
            scanf("%d", &mat[i * size + j]);
```

```
        }
    }
}

// Completarea array-ului cu valori random
void random_fill(int *arr, int size) {
    srand(time(NULL));
    for(int i = 0; i < size; i ++) {
        arr[i] = rand() % 100;
    }
}

void random_fill_mat(int *mat, int size) {
    srand(time(NULL));
    for(int i = 0; i < size; i++) {
        for(int j = 0; j < size; j++) {
            mat[i * size + j] = rand() % 100;
        }
    }
}

#pragma endregion

#pragma region Sortari
// HEAPSORT -> Crescator
void heapify(int *arr, int size, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if(left < size && *(arr + left) > *(arr + largest)) largest =
left;
    if(right < size && *(arr + right) > *(arr + largest)) largest =
right;

    if(largest != i) {
        swap(arr + i, arr + largest);

        heapify(arr, size, largest);
    }
}

void heap_sort(int *arr, int size) {
    for(int i = size / 2 - 1; i >= 0; i--) heapify(arr, size, i);

    for(int i = size - 1; i >= 0; i--) {
        swap(arr, arr + i);
        heapify(arr, i, 0);
```

```
    }
}

// COUNTING SORT -> Descrescator
void counting_sort(int arr[], int size) {
    int i, max = arr[0];

    for(int i = 1; i < size; i++) {
        if(arr[i] > max) max = arr[i];
    }

    int bucket[max + 1];
    for(i = 0; i <= max; i++) bucket[i] = 0;
    for(i = 0; i < size; i++) bucket[arr[i]]++;

    int j = 0;
    for(i = max; i >= 0; i--) {
        while(bucket[i] > 0) {
            arr[j++] = i;
            bucket[i]--;
        }
    }
}

// RADIX SORT -> Descrescator
void radix_sort(int arr[], int size) {
    int i, m = arr[0], exp = 1;
    int bucket[size], b[size];

    for(i = 0; i < size; i++) {
        if(arr[i] > m) m = arr[i];
    }

    while(m / exp > 0) {
        int bucket_count[10] = {0};
        for(i = 0; i < size; i++) bucket_count[9 - arr[i] / exp %
10]++;
        for(i = 1; i < 10; i++) bucket_count[i]  += bucket_count[i -
1];
        for(i = size - 1; i >= 0; i--) b[--bucket_count[9 - arr[i] /
exp % 10]] = arr[i];
        for(i = 0; i < size; i++) arr[i] = b[i];

        exp *= 10;
    }
}

// COMBSORT -> Crescator
```

```
void comb_sort(int *arr, int size) {
    int gap = size;
    int i, j, temp;
    int swapped = 1;

    while(gap > 1 || swapped) {
        gap = gap * 10 / 13;

        if(gap == 9 || gap == 10) gap = 11;
        if(gap < 1) gap = 1;

        swapped = 0;
        for(i = 0, j = gap; j < size; i++, j++) {
            if(*(arr + i) > *(arr + j)) {
                swap(arr + i, arr + j);
                swapped = 1;
            }
        }
    }
}

// MERGE SORT -> Crescator
void merge(int arr[], int left, int mid, int right) {
    int i, j, k;
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int Left[n1], Right[n2];

    for(i = 0; i < n1; i++) Left[i] = arr[left + i];
    for(j = 0; j < n2; j++) Right[j] = arr[mid + 1 + j];

    i = 0, j = 0, k = left;
    while(i < n1 && j < n2) {
        if(Left[i] <= Right[j]) {
            arr[k] = Left[i];
            i++;
        } else {
            arr[k] = Right[j];
            j++;
        }
        k++;
    }

    while(i < n1) {
        arr[k] = Left[i];
        i++;
        k++;
```

```
    }

    while(j < n2) {
        arr[k] = Right[j];
        j++;
        k++;
    }
}

void mergeSort(int arr[], int left, int right) {
    if(left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

// BUBBLE SORT -> Descrescator
void bubble_sort(int *arr, int size) {
    int i, j, temp;

    for(i = 0; i < size - 1; i++) {
        for(j = 0; j < size - i - 1; j++) {
            if(*(arr + j) < *(arr + j + 1))
                swap(arr + j, arr + j + 1);
        }
    }
}

// QUICK SORT -> Crescator
int partition(int arr[], int low, int high) {
    int pivot_index = low + (rand() % (high - low));
    if(pivot_index != high) swap(&arr[pivot_index], &arr[high]);

    int pivot_value = arr[high];
    int i = low;

    for(int j = low; j < high; j++) {
        if(arr[j] <= pivot_value) {
            swap(&arr[i], &arr[j]);
            i++;
        }
    }

    swap(&arr[i], &arr[high]);
```

```
    return i;
}

void quicksort_recursion(int arr[], int low, int high) {
    if(low < high) {
        int pivot_index = partition(arr, low, high);
        quicksort_recursion(arr, low, pivot_index - 1);
        quicksort_recursion(arr, pivot_index + 1, high);
    }
}

void quicksort(int arr[], int size) {
    srand(time(NULL));
    quicksort_recursion(arr, 0, size - 1);
}

// SHELL SORT -> Descrescator
int shell(int arr[], int size) {
    for(int i = size / 2; i > 0; i /= 2) {
        for(int j = i; j < size; j += 1) {
            int temp = arr[j];
            int k;
            for(k = j; k >= i && arr[k - i] < temp; k -= i)
                arr[k] = arr[k - i];

            arr[k] = temp;
        }
    }

    return 0;
}

// SELECTION SORT -> Crescator
int selection_sort(int *arr, int size) {
    for(int i = 0; i < size - 1; i++) {
        int min = i;
        for(int j = i + 1; j < size; j++) {
            if(*(arr + j) < *(arr + min)) min = j;
        }

        if(min != i)
            swap(arr + i, arr + min);
    }
}

// INSERTION SORT -> Descrescator
void insertion_sort(int arr[], int size) {
    int i, key, j;
```

```
    for(i = 1; i < size; i++) {
        key = arr[i];
        j = i - 1;

        while(j >= 0 && arr[j] < key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}

#pragma endregion

#pragma region Variante

void v8_1A(int *arr, int size) {
    int negPar = 0;

    // Alocarea memoriei pentru array-ul clona si salvarea datelor in
el
    int *clone = (int*)malloc(size * sizeof(int));
    clone_arr(arr, clone, size);

    // Conditia: primul element, numar negativ, pe pozitie para
    for(int i = 0; i < size; i++) {
        if(arr[i] < 0 && i % 2 == 0) {
            negPar = 1;
            break;
        } else negPar = 0;
    }

    if(negPar) heap_sort(arr, size);
    else counting_sort(arr, size);

    // Afisarea array-ului original (clone) si array-ului manipulat
(arr)
    print_arr(arr, clone, size);

    // Eliberarea din memorie al array-ului 'clone'
    free(clone);
}

void v8_1B(int *arr, int size) {
    int sumPar = 0, sumImp = 0, impNum = 0, medImp;

    // Alocarea memoriei pentru array-ul clona si salvarea datelor in
el
```

```
    int *clone = (int*)malloc(size * sizeof(int));
    clone_arr(arr, clone, size);

    // Conditia; suma elementelor pare mai mica decat media
aritmetica a elementelor impare
    for(int i = 0; i < size; i++) {
        if(arr[i] % 2 == 0) sumPar += arr[i];
        else {
            sumImp += arr[i];
            impNum++;
        }
    }

    medImp = sumImp / impNum;

    if(sumPar < medImp)
        radix_sort(arr, size);
    else
        comb_sort(arr, size);

    // Afisarea array-ului original (clone) si array-ului manipulat
(arr)
    print_arr(arr, clone, size);

    // Eliberarea din memorie al array-ului 'clone'
    free(clone);
}

void v8_1C(int *arr, int size) {
    int primes = 0;

    // Alocarea memoriei pentru array-ul clona si salvarea datelor in
el
    int *clone = (int*)malloc(size * sizeof(int));
    clone_arr(arr, clone, size);

    // Conditia: daca exista cel putin doua elemente numere prime
    for(int i = 0; i < size; i++)
        if(isPrime(arr[i])) primes++;

    if(primes > 1) {
        mergeSort(arr, 0, size - 1);
    } else {
        bubble_sort(arr, size);
    }

    // Afisarea array-ului original (clone) si array-ului manipulat
(arr)
```

```
    print_arr(arr, clone, size);

    // Eliberarea din memorie al array-ului 'clone'
    free(clone);
}

void v8_2A(int *mat, int size, int *arr) {
    int prod = 1;

    // Alocarea memoriei pentru matrice clona si salvarea datelor in
ea
    int *clone = (int*)malloc(size * size * sizeof(int));
    clone_mat(mat, clone, size);

    // Conditia: Daca produsul elementelor de pe prima coloana este
mai mare decat valoarea lui k...
    for(int i = 0; i < size; i++)
        prod *= mat[i * size];

    if(prod >= k) {
        // Diagonala principala
        for(int i = 0; i < size; i++)
            arr[i] = mat[i * size + i];

        quicksort(arr, size);

        for(int i = 0; i < size; i++)
            mat[i * size + i] = arr[i];
    } else {
        // Diagonala secundara
        for(int i = 0; i < size; i++)
            arr[i] = mat[i * size + (size - 1 - i)];

        shell(arr, size);

        for(int i = 0; i < size; i++)
        mat[i * size + (size - 1 - i)] = arr[i];
    }

    // Afisarea matricei originale (clone) si matricei manipulate
(mat)
    print_mat(mat, clone, size);

    // Eliberarea din memorie al matricei 'clone'
    free(clone);
}

void v8_2B(int *mat, int size, int *arr) {
```

```
    int isNegOnOdd = 0, isLowest = 0, line, lowestLine, min = 999;

    // Alocarea memoriei pentru matrice clona si salvarea datelor in
ea
    int *clone = (int*)malloc(size * size * sizeof(int));
    clone_mat(mat, clone, size);

    // Conditie: Daca elementul minim din array se afla cel putin
odata pe linie impara
    for(int i = 0; i < size; i++) {
        for(int j = 0; j < size; j++) {
            if(mat[i * size + j] < 0 && i % 2 == 1) {
                isNegOnOdd = 1;
                line = i;
                printf("%d on %d\n", mat[i * size + j], i);
            }
            if(mat[i * size + j] <= min) {
                min = mat[i * size + j];
                isLowest = 1;
                lowestLine = i;
            }
        }
    }

    if(isNegOnOdd) {
        for(int j = 0; j < size; j++)
            arr[j] = mat[line * size + j];

        selection_sort(arr, size);

        for(int j = 0; j < size; j++)
            mat[line * size + j] = arr[j];
    } else {
        if(isLowest)
            for(int j = 0; j < size; j++)
                arr[j] = mat[lowestLine * size + j];

        insertion_sort(arr, size);

        for(int j = 0; j < size; j++)
            mat[lowestLine * size + j] = arr[j];
    }

    // Afisarea matricei originale (clone) si matricei manipulate
(mat)
    print_mat(mat, clone, size);

    // Eliberarea din memorie al matricei 'clone'
```

```
    free(clone);
}

#pragma endregion

// Meniu
void menu(int *arr, int *mat, int size) {
    int option;

    // Printarea variantelor de problema
    printf("\nMENIU\n");

    printf("\n1) V8.1 - Alocarea memoriei\n");
    printf("2) V8.1 - Introducerea valorilor de la tastatura\n");
    printf("3) V8.1 - Completarea array-ului cu valori random\n");
    printf("4) V8.1 - Subprogramul A\n");
    printf("5) V8.1 - Subprogramul B\n");
    printf("6) V8.1 - Subprogramul C\n");

    printf("\n7) V8.2 - Alocarea memoriei\n");
    printf("8) V8.2 - Introducerea valorilor de la tastatura\n");
    printf("9) V8.2 - Completarea array-ului cu valori random\n");
    printf("10) V8.2 - Subprogramul A\n");
    printf("11) V8.2 - Subprogramul B\n");

    printf("\n99) Eliberarea memoriei\n");
    printf("\n0) Finisarea programului\n");
    printf("\nAlegeti programul: ");
    scanf("%d", &option);

    switch(option) {
        case 1:
            clrscr();
            while(size < 0 || size % 2 != 1) {
                printf("Introduceti dimensiunea array-ului: ");
                scanf("%d", &size);
            }

            arr = mem_alloc(size);
            break;

        case 2:
            clrscr();
            if(arr == NULL)
                printf("Error! Array-ul nu a fost alocat.\n");
            else
                input8_1(arr, size);
            break;
```

```
        case 3:
            clrscr();
            if(arr == NULL)
                printf("Error! Array-ul nu a fost alocat.\n");
            else
                random_fill(arr, size);
            break;

        case 4:
            clrscr();
            if(arr == NULL)
                printf("Error! Array-ul nu a fost alocat.\n");
            else
                v8_1A(arr, size);
            break;

        case 5:
            clrscr();
            if(arr == NULL)
                printf("Error! Array-ul nu a fost alocat.\n");
            else
                v8_1B(arr, size);
            break;

        case 6:
            clrscr();
            if(arr == NULL)
                printf("Error! Array-ul nu a fost alocat.\n");
            else
                v8_1C(arr, size);
            break;

        case 7:
            clrscr();
            while(size < 0) {
                printf("Introduceti dimensiunea matricei: ");
                scanf("%d", &size);
            }

            mat = mem_alloc_mat(size);
            arr = mem_alloc(size);
            break;

        case 8:
            clrscr();
            if(mat == NULL)
                printf("Error! Array-ul nu a fost alocat.\n");
```

```
        else
            input8_2(mat, size);
        break;

    case 9:
        clrscr();
        if(mat == NULL)
            printf("Error! Array-ul nu a fost alocat.\n");
        else
            random_fill_mat(mat, size);
        break;

    case 10:
        clrscr();
        if(mat == NULL || arr == NULL)
            printf("Error! Array-ul nu a fost alocat.\n");
        else
            v8_2A(mat, size, arr);
        break;

    case 11:
        clrscr();
        if(mat == NULL || arr == NULL)
            printf("Error! Array-ul nu a fost alocat.\n");
        else
            v8_2B(mat, size, arr);
        break;

    case 99:
        clrscr();
        if(arr == NULL && mat == NULL)
            printf("Error! Array-ul nu a fost alocat.\n");
        else free_mem(arr, mat);
        break;
    case 0:
        exit(0);
    default:
        clrscr();
        printf("Invalid subprogram!\n");
        break;
    }

    menu(arr, mat, size);
}

int main() {
    // Declararea array-ului si matricei
    int *arr = NULL, *mat = NULL;
```

```
    int size = -1;

    printf("Laborator 1 - SDA\n");

    menu(arr, mat, size);

    return 0;
}
```

Output:

Meniul:

```
Laborator 1 - SDA

MENIU

1) V8.1 - Alocarea memoriei
2) V8.1 - Introducerea valorilor de la tastatura
3) V8.1 - Completarea array-ului cu valori random
4) V8.1 - Subprogramul A
5) V8.1 - Subprogramul B
6) V8.1 - Subprogramul C

7) V8.2 - Alocarea memoriei
8) V8.2 - Introducerea valorilor de la tastatura
9) V8.2 - Completarea array-ului cu valori random
10) V8.2 - Subprogramul A
11) V8.2 - Subprogramul B

99) Eliberarea memoriei

0) Finisarea programului

Alegeti programul:
```

Afișarea array-ului original si cel manipulat:

```
Array-ul original: 35 30 65 25 44
Elementele array-ului dupa manipulare: 65 44 35 30 25
```

Introducerea valorilor de la tastatură:

```
Dati valoare elementului 0: 7
Dati valoare elementului 1: 4
Dati valoare elementului 2: 01
Dati valoare elementului 3: 10
Dati valoare elementului 4: -5
```

Concluzie:

În urma realizării acestei lucrări de laborator, ne-am adaptat pentru programarea in programe cu sute de linii de cod. Tot odată am aplicat 10 metode de sortare atât intru-un array unidimensional, cât și intr-un array bidimensional (matrice), crearea unu meniu recursiv, alocarea dinamică a memoriei pentru array și matrice și eliberarea memoriei după finisarea programului.

În această lucrare de laborator, am aplicat diferite algoritme pentru a valida condiției problemei, pentru a ajunge la un rezultat pozitiv și am practicat creare funcțiilor proprii pentru a ușura scrierea codului pentru program.